4. MDP를 알 때의 플래닝

- 문제가 작은 문제이고 MDP를 알 때의 경우를 다룸
- 테이블 기반 방법론(tatular method): 모든 상태 s 혹은 상태와 액션의 페어(s, a)에 대한 테이블을 만들어서 값을 기록해놓고, 그 값을 조금씩 업데이트하는 방식

4.1. 밸류 평가하기: 반복적 정책 평가

출발	s_1	s_2	s_3
s_4	s_5	s_6	s_7
s_8	s_9	s_{10}	s_{11}
s_{12}	s_{13}	s_{14}	종료

보상 : 스텝마다 -1

정책 : 4방향 uniform random

| 그림 4-1 | 다시 돌아온 그리드 월드

- 4방향 랜덤하게 움직이는 에이전트가 있고, 스텝마다 -1의 보상을 받을 때 각 상태의 밸류를 평가
- 종료까지의 누적 보상은 곧 에이전트가 걷게되는 총 걸음 수와 같은데 에이전트가 마치 술에 취한 사람처럼 완전히 임의로 움직이기 때문에 평균적으로 몇 걸음이 걸릴지 예상하기 어려움
- 반복적 정책 평가 사용

1. 테이블 초기화

s_0	s_1	s_2	s_3
s_4	s_5	s_6	s_7
s_8	s_9	s_{10}	s_{11}
s_{12}	s_{13}	s_{14}	종료

→

0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0

KYOBO eBook 6221025254 2010.2

| 그림 4-2 | 테이블 초기화

- 16개의 빈칸의 값을 0으로 초기화

2. 한 상태의 값을 업데이트

0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0

→

0.0	0.0	0.0	0.0
0.0	-1.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0

| 그림 4-3 | 벨만 기대 방정식을 이용해 파란색 상태의 값을 업데이트

- s_5를 정확하게 바꾸기 -> MDP 이용

$$v_\pi(s) = \sum_{a \in A} \pi(a \mid s)\left(r_s^a + \gamma \sum_{s' \in S} P_{ss'}^a v_\pi(s')\right)$$

- $v_\pi(s)$: 테이블에 담겨있는 임의의 값
- r_s^a: 환경이 주는 실제 값
- $v_\pi(s')$: 테이블에 담겨있는 임의의
- 노란 색 상태들의 값을 통해 파란색 상태의 값을 업데이트

$$v_\pi(s_5) = 0.25 * (-1 + 0.0) + 0.25 * (-1 + 0.0) + 0.25(-1 + 0.0) + 0.25 * (-1 * 0.0) = -1.0$$

- 맨 마지막 값(종료상태)는 항상 0
  - 미래에 받게 될 누적 보상의 기댓값으로 맨 마지막 상태의 입장에서는 그 뒤의 미래라는것이 존재하지 않음

3. 모든 상태에 대해 2의 과정을 적용

0.0	0.0	0.0	0.0
0.0	-1.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0

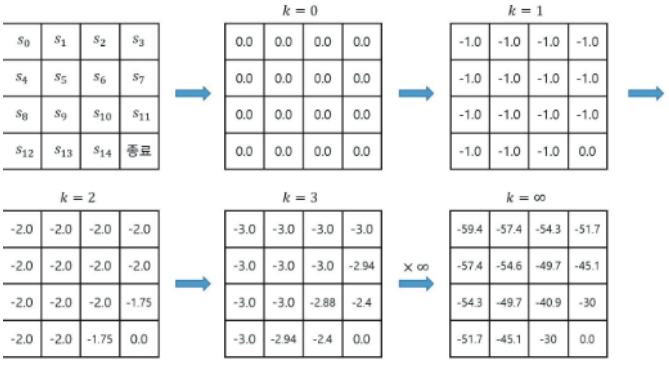
→

-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	0.0

| 그림 4-4 | 모든 상태의 값을 업데이트 하고 난 모습

- s_5의 값을 업데이트하고, 같은 방법으로 마지막 상태를 제외한 15개의 상태를 업데이트

4. 앞의 (2)~(3)의 과정을 계속해서 반복



| 그림 4-5 | 값이 수렴할 때까지 같은 프로세스를 반복

- 이 과정들을 계속 반복하면 실제 value에 수렴

4.2. 최고의 정책 찾기 – 정책 이터레이션

$k = \infty$

-59.4	-57.4	-54.3	-51.7
-57.4	-54.6	-49.7	-45.1
-54.3	-49.7	-40.9	-30
-51.7	-45.1	-30	0.0

그리디 정책

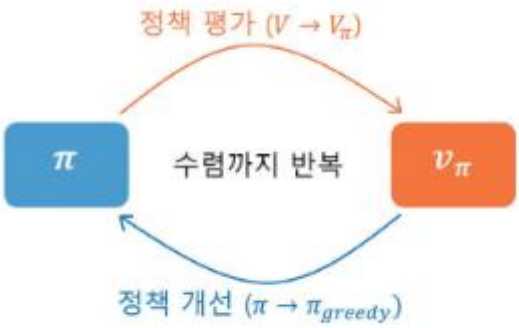
출발			
	↓→		
			종료

| 그림 4-6 | 가치 함수를 통한 파란색 상태의 그리디 정책 계산

- s_5의 가치는 -54.6이면 s_5에서 동쪽이나 남쪽(-49.7)로 가는 게 이득
    - 그리디 정책(greedy policy): 그저 눈 앞의 이익을 최대화하는 선택을 취하는 방식
    - 랜덤 정책 π에 대해 밸류를 평가하고 그에 대해 그리디한 정책을 표시하면 그 정책이 곧 최적 정책과 일

$$\pi'(a_{east} \mid s_5) = 1.0 \; or \; \pi'(a_{south} \mid s_5) = 1.0$$

평가와 개선의 반복



| 그림 4-8 | 정책 이터레이션의 도식

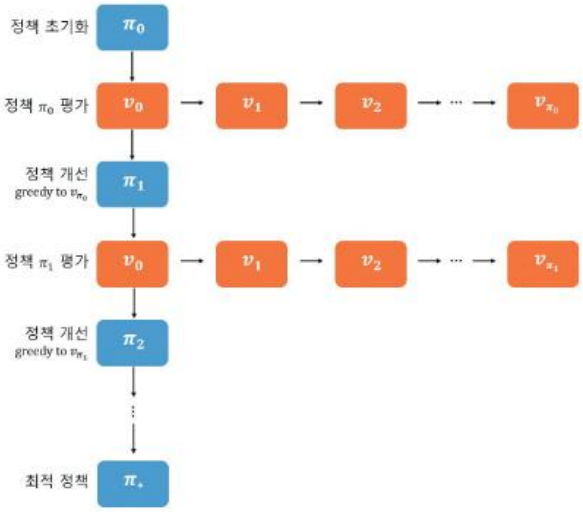
- 정책 이터레이션은 정책 평가(policy evaluation)과 정책 개선(policy improvement)로 구성
- 정책 평가: 반복적 정책평가.
    - 고정된 π에 대해 각 밸류를 구하고 π를 따랐을 때 각 상태의 가치를 평가
- 정책 개선: 그리디 정책 생성
    - 새로운 정책 π'를 생성

과연 정책은 개선되는가?

- 그리디 정책이 더 나은 정책으로 이어지지 않는다면 모든 과정은 말짱 도루묵
    - 다행히 그리디 정책은 기존 정책보다 좋다는 것이 보장됨
- 그리디 정책은 눈 앞에 더 큰 이익을 따라가는 정책
    - $\rightarrow \pi_{greedy}$가 π보다 높은 정책이 보장

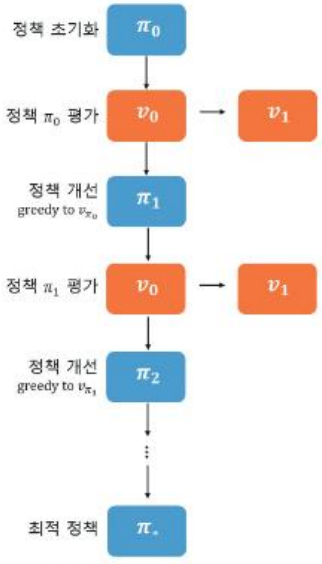
정책 평가 부분을 간소화하기¶

- 정책 이터레이션은 루프 안에 루프가 있기 때문에 많은 연산 필요
- 그리드 월드의 경우 문제가 간단해서 한바퀴 만으로 최적 정책을 얻을 수 있지만 그렇지 않은 경우가 많음



| 그림 4-9 | 정책 이터레이션을 풀어서 나타낸 그림

- 위 루프에서 가장 많이 필요로 하는 단계는 "정책 개선 단계"
- 반복적 정책 평가를 많이 반복, 심지어는 무한까지 반복하는건 현실적으로 불가능
- 이 과정을 조금 일찍(즉 정확한 가치를 평가할 수 있을때까지만) 멈춰도 됨

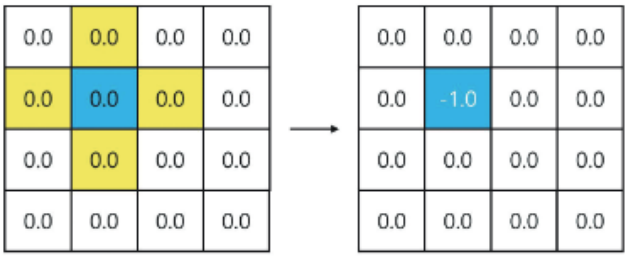


| 그림 4-11 | 간결해진 정책 이터레이션

- 즉 가치함수를 끝까지 학습하지 않아도 됨. 오로지 해당 단계에서 그리디 정책이 현재 정책과 다르기만 하면 됨

4.3. 최고의 정책 찾기: 밸류 이터레이션

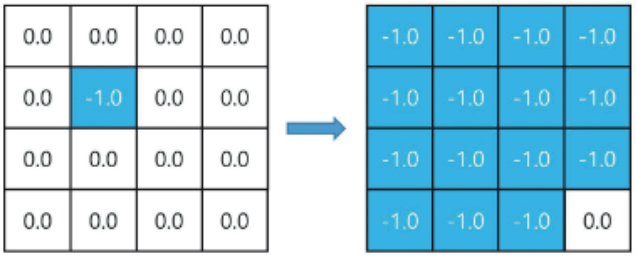
- 밸류 이터레이션(value iteration): 오로지 최적 정책이 만들어내는 최적 밸류 하나만 보고 달려감
- 정책 이터레이션에서는 테이블의 값들이 해당 평가 단계에서 사용하는 정책 π의 밸류
    - 최적정책에선 π^*의 밸류인 $v_*(s)$를 뜻함



| 그림 4-13 | s_5의 값을 업데이트한 결과

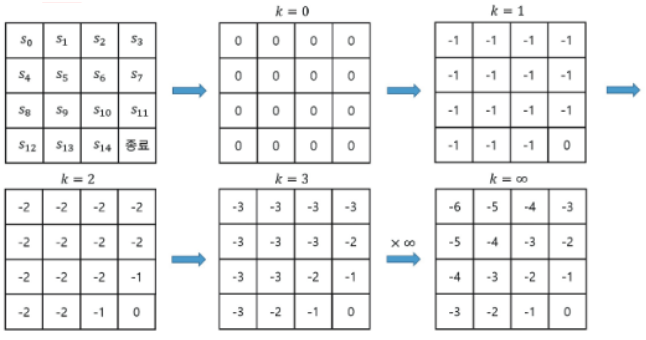
- s_5에서 시작해서, 벨만 최적 방정식을 적용

$$v_*(s) = \max_a \left[r_s^a + \gamma \sum_{s' \in S} P_{ss'}^a v_*(s') \right] r_s^a = -1, \gamma = 1, P_{ss'}^a = 1 \therefore v_*(s_5) = -1.0$$



| 그림 4-14 | 모든 상태에 대해 벨만 최적 방정식을 적용

- 다음 상태 s'에 대해 $v_*(s')$의 값은 모두 -1로 같음
    - 다만 종료 상태에선 더이상 받을 보상이 없어서 0으로 고정
- 이를 계속 반복하면 특정 값에 수렴함. 수렴한 값은 즉 각 상태의 최적 밸류
- 시작지점인s_0에서 종료상태까지 6걸음이 필요



| 그림 4-15 | 수렴할 때까지 반복하고 난 후의 결과

- 그러나 우리는 최적 밸류가 아닌 최적 정책을 찾는 것